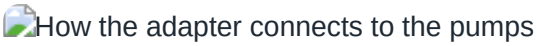
ioBroker.pondpump

User Manual — set up, control and monitor your OASE AquaMax Eco Titanium pond pumps in ioBroker

Beginner-friendly guide

1. What this adapter does



How ioBroker reaches your pumps: today via the OASE cloud, and — in local mode — directly over your LAN.

The **pondpump** adapter connects ioBroker to your **OASE AquaMax Eco Titanium** pond pump(s) through the **OASE Garden Controller Cloud (EGC)** gateway. Once it is running you can, from ioBroker (and therefore from VIS, scripts, scenes, Alexa, etc.):

- **switch each pump on and off**,
- **set the pump speed** from 0 – 100 %,
- **read live telemetry**: power (W), motor speed (rpm), water/electronics temperature (°C) and mains voltage (V),
- **see the connection and device status**.

Each pump keeps the name you gave it in the OASE app (e.g. *Waterfall*, *Filter*), so you can recognise it in the object tree.

> 💡 **Good to know:** the adapter talks to the **controller** (item 55317), which in turn talks to the pumps (item 73656). It is a separate project from the community adapter for the OASE socket controllers — it targets the smart pond pumps.

---

2. Before you start — what you need

You need	Why
A running **ioBroker** installation (js-controller, Node.js ≥ 22)	The platform this adapter runs on
An **OASE Garden Controller Cloud** (EGC, item 55317), set up in the OASE app	The gateway the adapter connects to
One or two **OASE AquaMax Eco Titanium** pumps (item 73656), paired in the app	The devices being controlled
Your pumps **already working in the OASE app**	The adapter uses the same cloud account
A **cloud refresh token** (see chapter 4)	How the adapter logs in without your password

> 💡 **Tip:** get everything working in the **OASE app first**. If the app can switch the pumps, the adapter can too.

---

3. Installing the adapter

The adapter is published on npm as `iobroker.pondpump`. Until it is part of the official ioBroker repository, install it from its source:

1. Open the ioBroker **Admin** UI.
2. Go to **Adapters** and switch on **Expert mode** (the wizard-hat icon, top right).
3. Click the **cat/octocat "Install from own URL"** icon.
4. Enter one of:
   - the npm package name `iobroker.pondpump`, or
   - the GitHub URL of a release tarball if you were given one.
5. Confirm and wait until the adapter appears in the list.
6. Click the **+** on the pondpump tile to create an **instance** (`pondpump.0`).

The instance configuration opens automatically. Leave it for a moment — first we need a refresh token (next chapter).

---

4. Getting a cloud refresh token (step by step with mitmproxy)

The OASE cloud uses **Microsoft Azure AD B2C** for login. For security the adapter does **not** store your account password. Instead it uses a **refresh token** — a long, one-time credential that your OASE app receives when it logs in. You capture that token **once** with a small tool called **mitmproxy**, paste it into the adapter, and from then on the adapter refreshes it automatically.

Don't worry if you've never done this — follow the steps below exactly.

mitmproxy sits between your phone and the OASE cloud, so you can read the login and copy the token.

4.1 Install and start mitmproxy

mitmproxy is a small, free program. We'll use its browser version, **mitmweb**. Follow the steps for **your** operating system.

Windows (using PowerShell)

1. Open your web browser and go to **https://mitmproxy.org/downloads/**.
2. Download the **Windows** package (the newest version — usually a `.msi` installer).
3. Open the downloaded file and click through the installer: **Next** → **Next** → **Install** → **Finish**.
4. Now open **PowerShell**:
   - Press the **Windows key**, type `PowerShell`, and click **Windows PowerShell** in the list.
   - A dark window with a blinking text cursor appears — this is the command line.
5. Type this command and press **Enter**:

   ```
   mitmweb
   ```

6. If Windows asks whether to **allow network access**, click **Allow**. Your browser opens a new tab at **http://127.0.0.1:8081** — that's the mitmproxy control panel. mitmproxy is now waiting for phone traffic on **port 8080**. ✅
7. **Keep this PowerShell window open** the whole time — closing it stops mitmproxy. Later, to stop it, click the window and press **Ctrl + C**.

> 💡 **"mitmweb is not recognized"?** Close PowerShell and open it again (so it notices the newly installed program). If you downloaded the `.zip` version instead of the installer, unzip it, then in PowerShell type `cd` followed by the folder path and run `.\mitmweb.exe`.

macOS

1. Open **Terminal** (press **Cmd + Space**, type `Terminal`, press Enter).
2. The easiest way is with Homebrew: run `brew install mitmproxy`. (No Homebrew? Download the macOS build from **https://mitmproxy.org/downloads/** and unzip it.)
3. Run `mitmweb`. A browser tab opens at **http://127.0.0.1:8081**.

Linux

1. Install it with `pipx install mitmproxy` (or your distribution's package, or the binaries from the downloads page).
2. Run `mitmweb` in a terminal and open **http://127.0.0.1:8081**.

In every case: the browser page on **:8081** is the control panel you'll watch, and **port 8080** is where your phone will send its traffic (next step).

4.2 Send your phone's traffic through mitmproxy

Your phone and computer must be on the **same Wi-Fi**.

1. Find your **computer's local IP** (e.g. `192.168.1.20`): Windows `ipconfig`, macOS/Linux `ip addr` / `ifconfig`.
2. On the phone: **Wi-Fi settings** → **your network** → **Proxy** → **Manual** and enter **Server = your computer's IP**, **Port = 8080**. Save.
3. Open the phone's browser and visit **http://mitm.it**. Choose your phone's system, **install** the offered certificate **and trust it**:
   - **iOS:** install the profile, then *Settings* → *General* → *About* → *Certificate Trust Settings* and turn the mitmproxy certificate **on**.
   - **Android:** install it as a **CA certificate** (Settings → Security → Encryption & credentials → Install a certificate → CA certificate).

   This certificate is what lets mitmproxy read the otherwise-encrypted OASE traffic. **Remove the proxy and the certificate again when you're done.**

4.3 Capture the login and grab the refresh token

1. In the **mitmweb** page, clear the list (so new requests are easy to spot).
2. In the **OASE app**: **log out**, then **log back in**.
3. In mitmweb's **filter box** at the top, type one of these to jump straight to the right request — this is the trick that saves you scrolling through hundreds of entries:

Type this filter	It shows
`~u token`	only requests whose URL contains "token"
`~d account.oase.com`	only requests to the OASE login server
`~b refresh_token`	only requests whose body contains `refresh_token`

4. Click the **POST** request that ends in `/oauth2/v2.0/token`.
5. Open its **Request** tab and look at the form body. Find `refresh_token=` and copy the long value after it (up to the next `&`).
   - **Extra tip:** press `/` in mitmweb and search for `refresh_token` to highlight it instantly.
6. Paste that value into the adapter setting **"Cloud refresh token"** (chapter 5).

> 💡 The refresh token is long (hundreds of characters) — copy **all** of it. Treat it like a password: don't share it. You can revoke it any time by logging out everywhere in the OASE app. **Your account password is never entered into the adapter.**

4.4 (Advanced) Find the device password for local mode

Only needed if you want connection mode `local` / `both` (chapter 8). While mitmproxy is still running:

1. In the app, open your pond so it loads the pumps (this triggers the inventory download).
2. In mitmweb's filter box, type `~u Inventory` to show the request to `/User/Inventory`.
3. Click it and open the **Response** tab. In the JSON, find the pump's **custom attributes**; the entry with `Id` **= 101** holds the **device password** — a **64-character** value (it may contain `\uXXXX` escape sequences, that's fine).
4. Copy that value into the adapter setting **"Device password"**. The adapter decodes it and uses it for the local TLS handshake.

> 💡 **If mitm.it won't load:** double-check the phone's proxy points at your computer's IP on port 8080, and that traffic is flowing. On iOS you must both **install** *and* **trust** the certificate (two separate steps).

---

5. Configuring the instance

Open **Instances** → **pondpump.0** → **settings** (the wrench icon). The settings are grouped:

Connection

Setting	What to enter
Connection mode	`cloud` for the normal (internet) path. `local` / `both` are for the in-house LAN path (chapter 8, experimental).
Poll interval	How often (seconds) the adapter reads status. Default **30**. Minimum 5.

Cloud

Setting	What to enter
Cloud refresh token	The token from chapter 4 (stored encrypted).
Advanced (base URL, token URL, client id, scope)	Leave at their defaults unless OASE changes their cloud.

Local (only for `local` / `both`)

Setting	What to enter
Controller IP	The EGC controller's address in your network.
Device password	The 64-character device password (advanced; see chapter 8).
Bind address / Port	The ioBroker host address and TCP port the controller connects back to (default 5999).

Click **Save**. The instance starts and, after a few seconds, `info.connection` should turn **true**.

---

6. The objects the adapter creates

After the first successful poll you will find these under `pondpump.0`:

```
pondpump.0
├── info.connection            (true when connected)
├── <gateway>                  the EGC controller (device)
│   ├── serialNumber, firmware
│   └── online                 (controller reachable)
└── pumps.<deviceNumber>       one device per pump, named after the app
    ├── control.on             ← switch on/off        (writable)
    ├── control.speed          ← speed 0–100 %        (writable)
    ├── control.speedRaw       ← speed 0–255 (raw)    (writable)
    ├── status.connected       pump reachable
    ├── status.fcStatus        controller status text
    └── telemetry
        ├── power              live power in W
        ├── speed              live motor speed in rpm
        ├── temperature        °C
        ├── temperature2       °C (second sensor)
        ├── voltage            mains voltage in V
        └── raw.sensorN        still-unclassified sensor values
```

---

7. Controlling and reading the pumps

Switch a pump on/off — set `pumps.<deviceNumber>.control.on` to `true` / `false`.

Set the speed — write a percentage (0–100) to `pumps.<deviceNumber>.control.speed`. The adapter sends the command, confirms it, and re-reads the pump shortly after so the states reflect reality.

Read telemetry — the values under `telemetry` update live on every poll (fast values like power and rpm each cycle, slower ones like temperature every few cycles to keep the cloud happy). Use them in VIS, charts, or scripts.

Example (JavaScript adapter):

```
// Run the "Waterfall" pump at 70 %
setState('pondpump.0.pumps.1234567.control.speed', 70);

// Log its live power
on('pondpump.0.pumps.1234567.telemetry.power', (obj) => {
    log('Pump power: ' + obj.state.val + ' W');
});
```

---

8. Local mode (in-house LAN) — experimental

The long-term goal is to control the pumps **without the internet**, directly over your LAN. The foundation is in place (connection mode `local`):

- ioBroker runs a small **TLS server**; the adapter sends a **UDP wake** packet to the controller, which then **connects back** and authenticates with the **device password**.

To try it you need the **controller's IP**, the **64-character device password**, and an open network path (UDP 5959 out to the controller, TCP 5999 back to ioBroker). Live pump data over the local channel is still being finished — for everyday use, stay on **`cloud`** mode for now.

---

9. Troubleshooting

Symptom	What to check
`info.connection` stays **false**	Is a **refresh token** entered? Capture a fresh one (chapter 4) — tokens can expire if you log in elsewhere.
Log says **AUTH FAILED**	The refresh token is invalid/expired → capture a new one.
No pumps appear	Are the pumps online in the **OASE app**? The adapter mirrors the cloud inventory.
Commands do nothing	Wait for the **first successful poll** (the adapter learns the pump addressing then). Check the log.
Want more detail	Set the instance **log level to** `debug` — every step is logged with a tag like `[poll]`, `[cloud/auth]`, `[cloud/cmd]`. Secrets are never logged.

The log lines are tagged by component so any problem can be pinpointed. When reporting an issue, include the debug log around the failure.

---

10. Privacy & safety

- Your **OASE account password** is never entered into or stored by the adapter.
- The **refresh token** and **device password** are stored **encrypted** in ioBroker.
- The adapter only talks to the OASE cloud (or, in local mode, directly to your controller).
- Use at your own risk — this is an unofficial community project, not affiliated with OASE GmbH.

Questions or problems? Open an issue at the project's GitHub repository. Happy pond-keeping! 🐟